

Web Architecture

Lecture-1

Work with backend

Build facebook from scratch

Install Java 21

APIs : connect frontend and backend

TCP : bidirectional connection
: unidirectional connection

API : (1) Bidirectional
(2) Unidirectional

Lecture-2

- ✓ 1. Request / Response
- ✓ 2. Method
- ✓ 3. Headers, status
- ✓ 4. Session/ cookies
- ✓ 5. MVC / layer

(2 types)

API

Frontend

Backend

Database

Web Socket

request body { text:
vdo:
Security:

}

Header:

↑
cookies

token: (Generalized) (Access token) (efg) expired
name:
country:

(security)

/refreshtoken

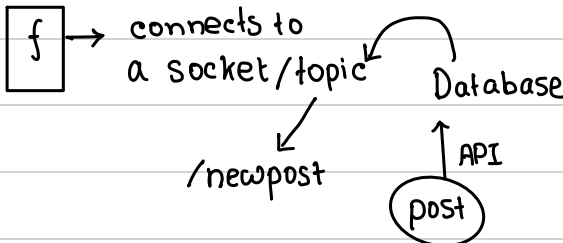
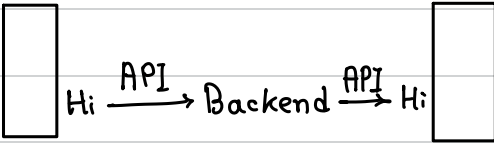
↳ username

↳ refreshtoken (abc)

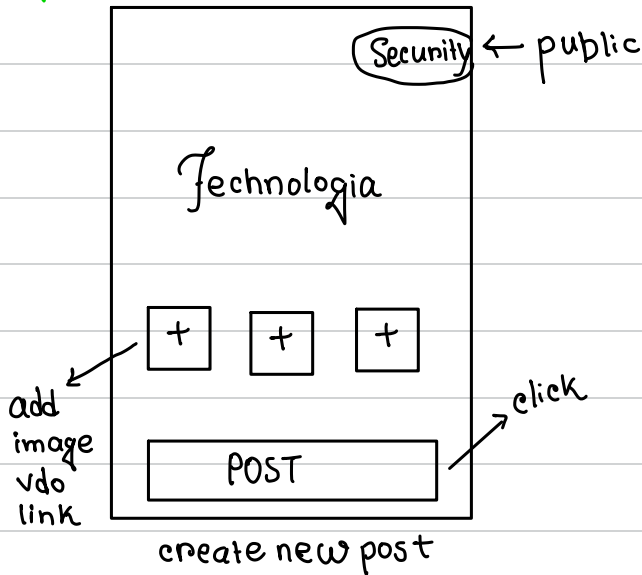
↳ accesstoken (efg)

Messenger:

U1 ↔ U2
channel



post something on facebook:



Ponichay application:

had facial expression + NID info

had poor security

First login in facebook:

1st login → username

password

2 step verification

↓ Success

gives : 1. access token (xyz) 5min

2. refresh token (abc) 300days

exe

Logout:

It deletes the refreshToken from database

Response:

Body : { Data :

JSON

Debug:

Times:

}

Status code : 200 , (success)

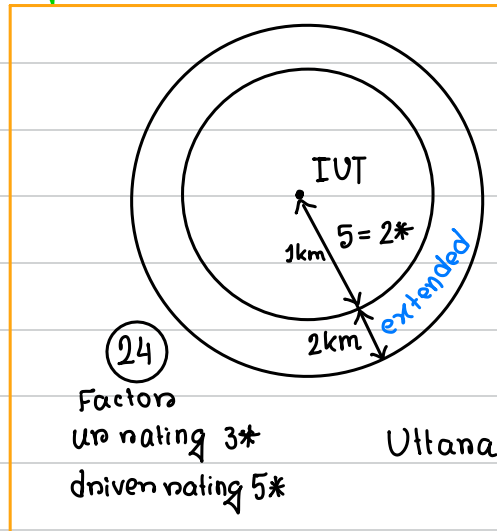
400 , (Bad request)

401 , (unauthorized)

404 , (not found)

500 , (Internal Server Error)

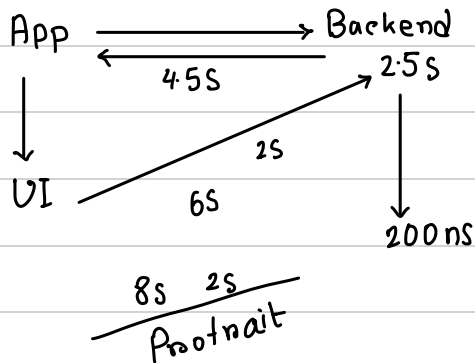
Driven Priority Engine:



HTTP/S:

SSL → security certificate

which verifies that the website is secured



Lecture-3

- ✓ 1. Rest API type
2. Tiered Architecture
- ✓ 3. MVC
- ✓ 4. SOC
- ✓ 5. Controller Service Repository
- ✓ 6. Coupling

Four types of rest API:

1. GET (no body)
2. POST (header + body)
3. PUT
4. DELETE

All API have
header + body

GET: `http://baseurl/getUser/{userid}` → path variable on parameter

POST: `http://baseurl/createUser`

body { id:

session:

HSC pass:

⋮

DOB: }

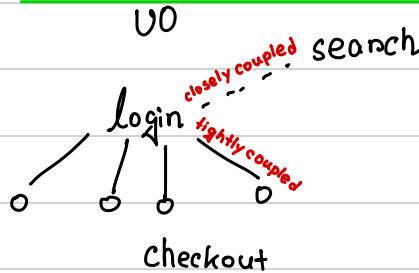
☞ We can do everything with POST api but why do we need GET, PUT or DELETE?

1. User onboard	6. Admin
2. login	7. Cart
3. Product enlist	8. Promocode
4. Searching and Sorting	9. Delivery
5. Checkout	

→ ξ_{commence}

Coupling (Dependency)

Checkout without
login - make it
loosely coupled

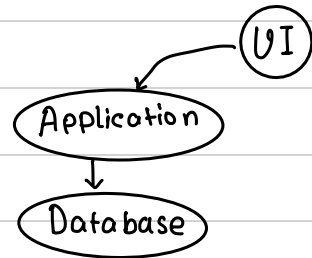
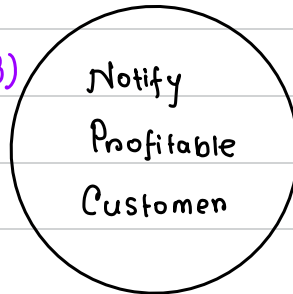


Validating / optional

2 layer - (app + DB)

3 layer (end user involved)

4 layer



Product listing

1.	Toyota car

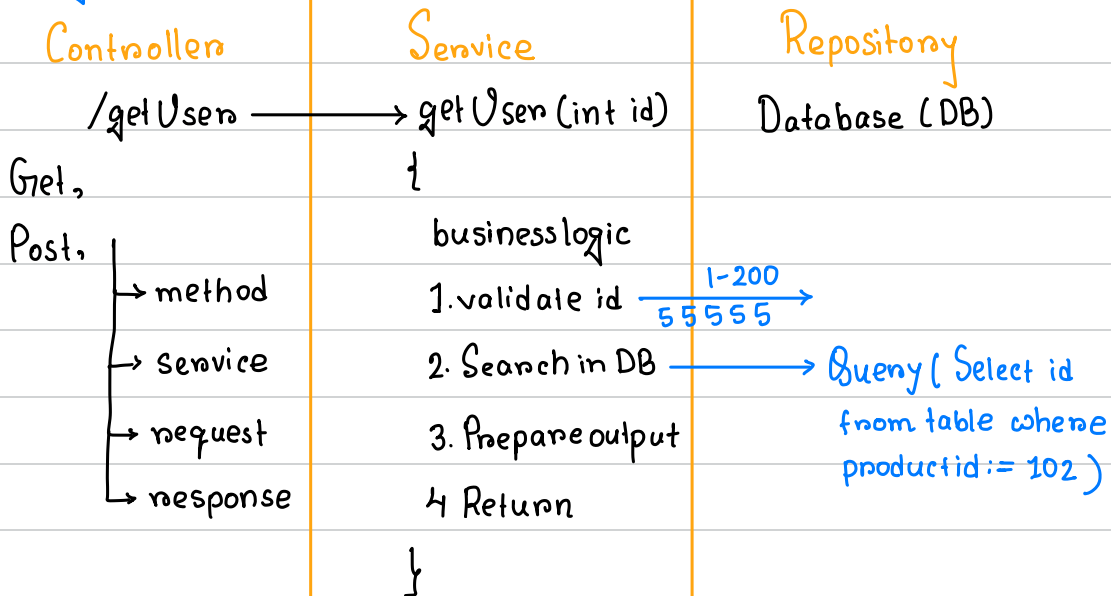
id	product	prefer user
1.	Toyota car	Sadia, Takiya

MVC
(Model View
Controller)

MVC

Model	View	Controller
Entities: • product • user • notifi • cant		• Service (logic) • Repository (Database)
DTO: • product DTO	<u>Validating:</u> make this optional to make checkout loosely coupled. So we have to know which feature could be loosely coupled in the future	

Spring Boot Framework:



Lecture-4

- ✓1. Bean
- ✓2. Lifecycle of Bean
- ✓3. Singleton
- ✓4. Thread
- ✓5. Pool (Bean, Thread)
- ✓6. Monolith vs Microservice
- ✓7. Background job

OOP:

- ✓1. Encapsulation
- ✓2. Inheritance
- ✓3. Abstraction
- ✓4. Polymorphism

☐ private methods can be passed to child class but
it can't modify private method

override — redefine method of parent class

overload — define multiple method in same class
same name diff parameter

ssl ecommence — public - private

Abstraction

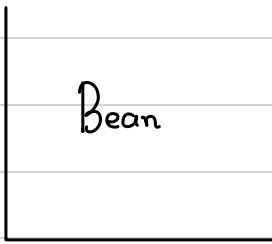
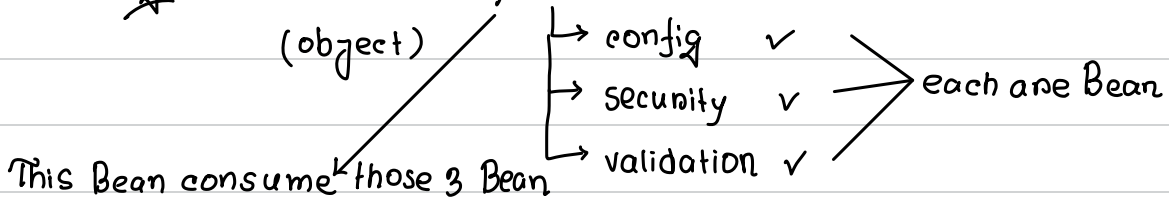
private

public

protected

Bean: each of the class is bean
(single most atomic object)

eg: A Bean is Payment.
(object)

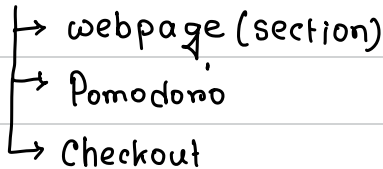


Bean Pool

Instantly create an object no need
to initialize again
all beans are created with definition

eg: Ecommerce

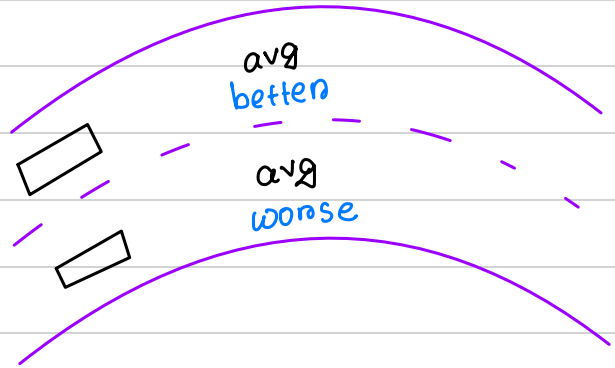
Price



Bean life cycle

Singleton class — create only one object
1 instance for entire app
can't initiated again

Threads:



Why we need Bean?

- ↳ In order to avoid unnecessary space, declare-create-destroy instead take instance from Bean pool use and return
Thuse eg. use 1MB instead of 5GB

Monolith vs Microservice:

<u>Supershop</u>	<u>Local</u>
Pro: all in one complex to find save Import cons: fixed price (usuall high) flond to organise ISS	Pro. cheaper easy to find Setup own produce cons: no AC messy dinty strikex ☐ Fish ✓ ☐ meat ✓ ☐ vege
monolith	microservice
all responsibility in one app	serving diff feature differently

"Web Lab"

FirstClassApplication ▷ → to run

maven-wrappers.properties : 10 lolipops in 1 big lolipop

.gitignore — which don't want to include in repo

mvnw

pom.xml → necessary info

dependency can be include

<artifactId> used for testing

Test folder → test cases

** application.properties → db

jdbc:mysql://localhost:3306/ root 715

postgres port - 5432

App type

stent ←	dev
Test ←	Stage
production ←	Prod

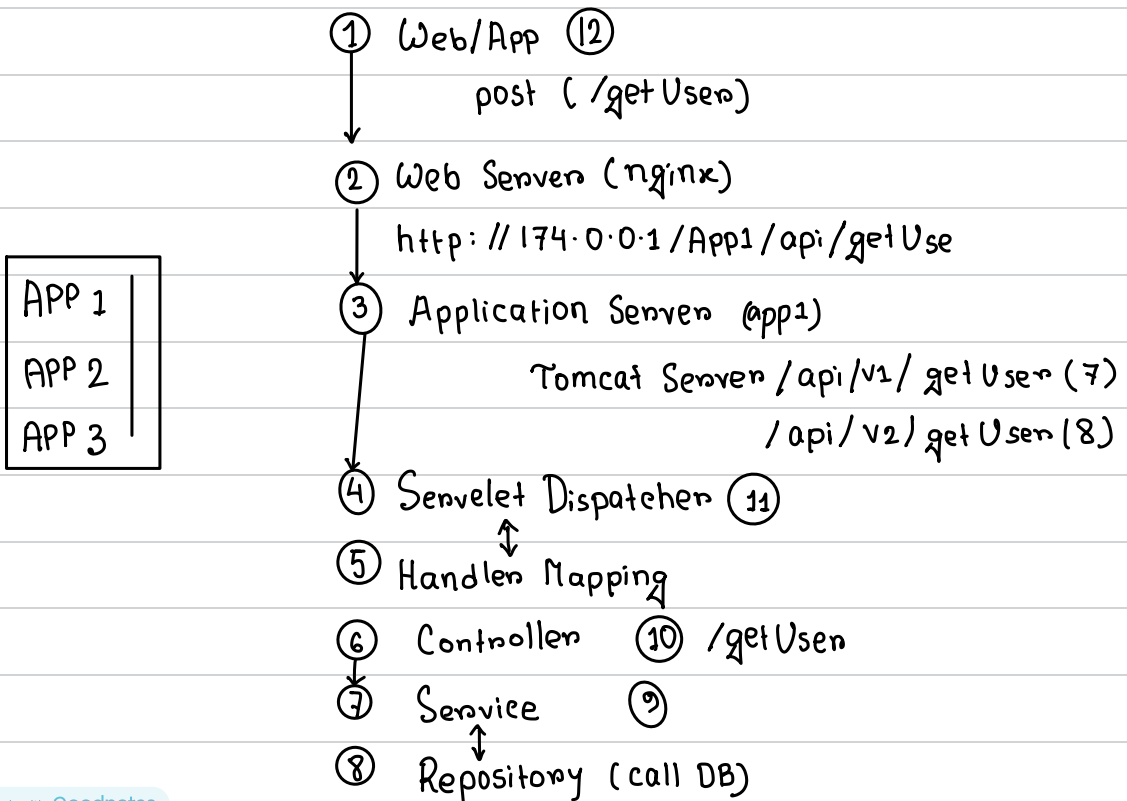
install maven locally

mvn clean

use mysql

Lecture - 5

- ✓ 1. Web / App
- ✓ 2. Web Server
- ✓ 3. Application Server
- ✓ 4. Dispatcher
- ✓ 5. Handler
- ✓ 6. Controller
- ✓ 7. Service
- ✓ 8. Repository



Background Job:

Select * from Student
where (Select id
from CSE
where id < 2000)
order by Name Asc

Release1 - 1st Jan

Release2 - 10th Jan

Quiz-1

23.05.25

Lecture (1-5)

limit offset

☐ All employee who is getting the 2nd highest Salary.

Select emp1* from Employee emp1
where Salary =
(Select Distinct (emp2.Salary)
from Employee emp2
order by Salary
Limit 1
Offset 2)

ID	name	salary
		200
		300
		250
		300
		250
		300

300	
300	
300	
250	
250	
200	
	<u>Distinct</u>
	300
	250
	200

max (max) (max) n3
max is a very operation
so avoid, it checks all row
all column and all value

offset 2
mean 0,1,2

“ pagination ”
1-10 in 1 pg
11-20 in 1 pg

Asynchronous vs Synchronous:

get popular Products ()

Background task

heavy operation

so do once/min

don't do always

when new user

arrive

Asynchronous: → Background job

(1) Signup

(2) auth

(3) email

(4) verify

(5) login

}

@async

→ done separately

may not in same time

Synchronous:

serially, sequentially

Quiz-1 (27.05.25)

- ✓ 1. Static /dynamic web
- ✓ 2. API (Rest)
- ✓ 3. Types of API
- ✓ 4. HTTP Methods
- ✓ 5. Header, Body
- ✓ 6. Token
- ✓ 7. Status code
- ✓ 8. Bean
- ✓ 9. OOP
- ✓ 10. Containers of Bean
- ✓ 11. Thread
- ✓ 12. Microservice vs Monolith

Lecture-6

1. Dependency Injection
2. Asynchronous - Synchronous

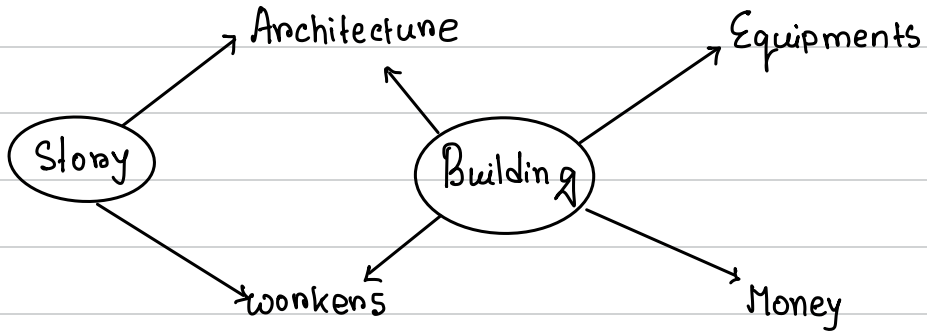
Synchronous: → uni
↓
submit paper
↓
confirmation
↓
pay
↓
admitted

Asynchronous: → apply
↓
allow to give xm
↓
admission test
↓
Result
↓
sc ←
→ Ad

6 classes:

4 component

2 service



We need to create instance of the components

```
Worker w1 = new Worker();
```

```
w2 ...
```

```
w3 ..... Garbage (no need to create everytime)
```

```
class Worker{
```

```
    long id;
```

```
    String Gender;
```

```
    Double Price;
```

```
    public Worker getWorker()
```

```
    {
```

```
}
```

Build 4 building

when work- is done

then assign to another building

~~A1~~

A2

~~w1~~

w2

work finish, destroyed

Complete OOP course

Submit a doc:

2 thing to do :

1. Abstraction's 2example (real life)
- 2 Inheritance's 2example

Lab → Microservice application

CRM - customer relationship management

Lecture - 7

1. Annotation
2. Security Configuration
3. Controller
4. Scheduled
5. Rest Controller

Annotation:

Spring boot Application

@configuration ^(N)	Scheduled	Component ^(N)	service
Enable auto configuration	Bean ^(N)		Controller
Component Scan	Transaction ^(N)		Repository
Enable Scheduling	Modifying ^(N)		Rest Controller
Spring Boot Test			Entity
			Column

Not Null	Request Mapping	valid	Autowired
Size	Get Mapping	Query	Qualifier
Pattern	Post Mapping	Validated	
JSON Ignore	PUT "		
Transient	Delete "		
	Request Param		
	Request Body		

Public Class getUsers (int id)

Date Validation:

dd/mm/yyyy

@Dateformat

date.validate()

private LocalDate DOB;

Security Configuration AWS S3:

- region
- access key
- secret key

@Scheduled (initial Delay
fixed Date = 5min "work automatically at fixed
fixed Delay = 1min time "

recalculate Attendance ()

5mins

Shondha

Student

ID	name	dept

class Student{

@ID

Integer ID;

@Column (name = "Name")

String ~~name~~; Student name;

@JsonIgnore

Department Dept

}

Q. When do you use controller and when we use Rest Controller ?

Lecture-8

1. Database

2. ERD

3. Cach

4. Relational DB

Database:

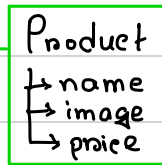
Monolith → 1 DB

Microservice → 1DB

1DB

1DB

Product



Search

Checkout

Wallet

Promo

Shint

Danaz

sellen

Buyer

1
2
3

A
d
m
i
n

4
5
6

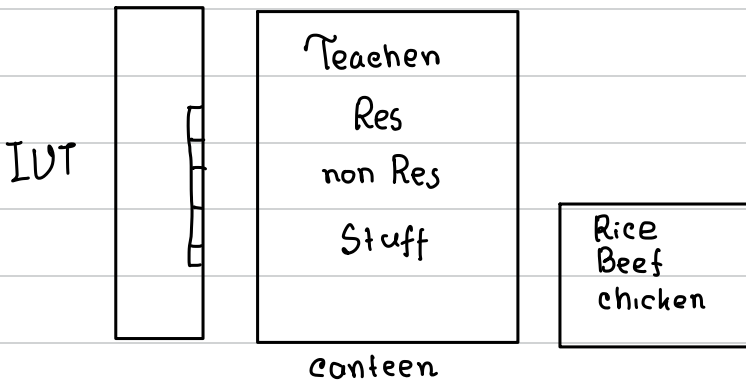
6 different
connection
App → DB

Separation of Concern:

Search — for Sellers

Product — for Buyers

- in order to decrease search time
- separating our app based on users



Why RDBMS over no sql?

↳ ensure ACID (Atomicity, Consistency, Isolation, Durability)

ideal for complex query

transactional system

ensure data integrity

Relationship

Q Difference between RDBMS and non RDBMS

↳ Searching is faster in non RDBMS

becz no' need to join table or find relations
and no need to consider all the constraints

Cache:

remember only 508 web archi class
then we can find a definite student
time complexity ↓

Relational DB:

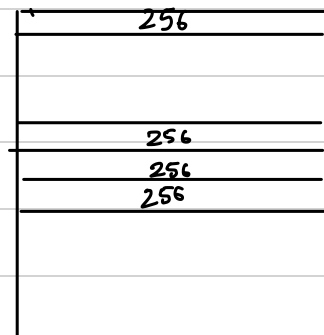
1. Postgres
2. MySQL
3. Sql Server
4. Oracle

Cache:

Redis
Guava
Finestone

1. Persistent data in RDBMS using id and store forever
stored in harddisk

2. cache → data is stored RAM/memory
update data frequently,



1GB used

[now need 3GB
continuous space

[defragmentation
reorganise the space
and clean 3GB
continuous space

Danax flash scale product
frequent visited data > stored in cache

persistant data like id → store in disk

TTL:

feature of cache

when data will automatically deleted in cache

Lecture-9

1. ORM
2. JPA
3. Event driven
4. Communication



```
public class Student () {  
    int id,  
    String name,  
    Dept dept  
}
```

Query:

Select * from Student;

Postgres - Oracle - MySql

If need to change db bcz other db might required shorter query then JPA makes it easier. It doesn't use direct query.

- 1 List <Entity> findAll();
2. findAllInIdOrderByNameDesc list <id>,

* It will automatically convert the ORM when migrate different db.

5marks for Quiz

<u>Student</u>	<u>Course</u>	<u>Lectures</u>	<u>Attendance</u>
id name courseid	id name	id name courseid	id studentid date lectureid isPresent

Q. All students over 80% attendance by course id - 13 ?

↳

Select s.id , s.name

ORM | (n+1)
JPA

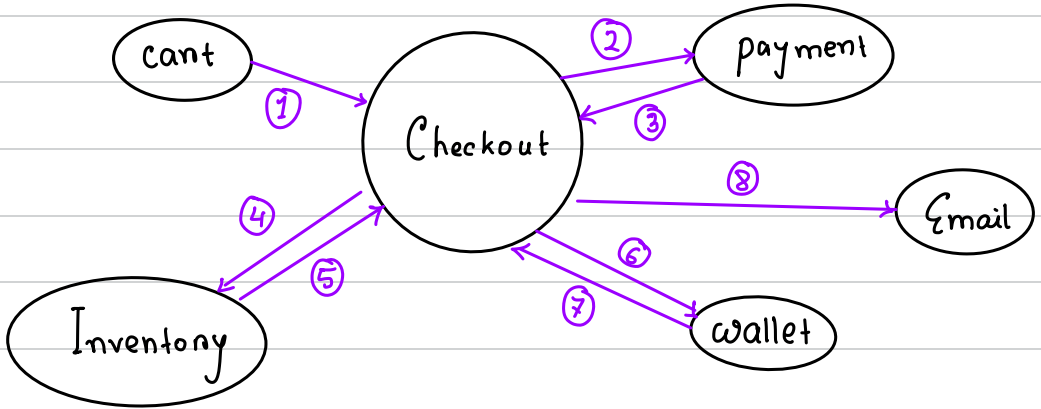
Select studentid from Attendance
where courseid = 13

group by studentid

Having avg(Case when isPresent Then 1
(else 0 End) > 0.80;

1. Inter Service Communication

Ecommerce



Problem Solving

1. Problem Solving
2. Solution planning
3. Code
4. Optimization

Google

Rest API / GRPC / Message Queue ***

Topic → Inverse Points

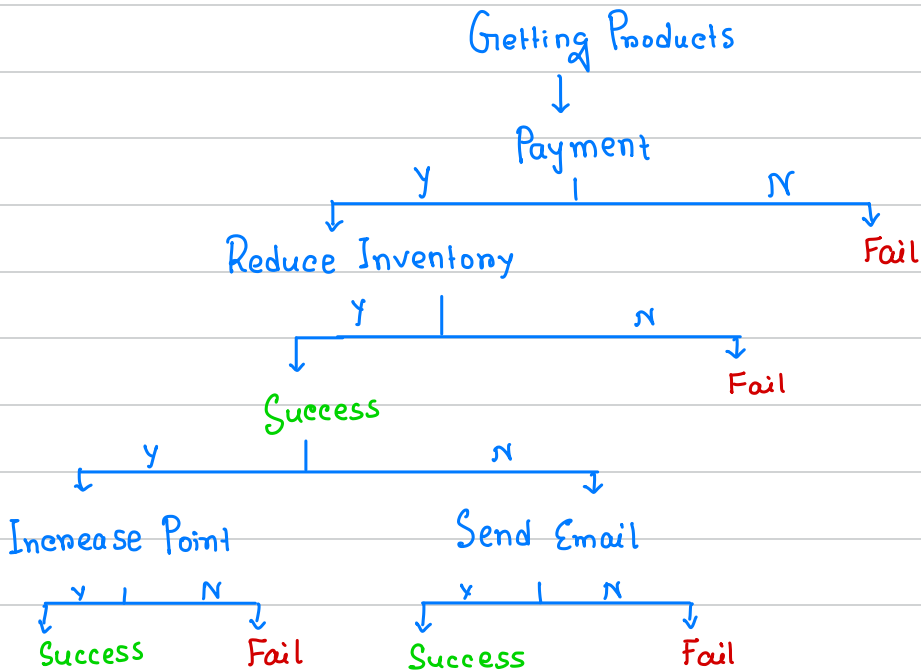
↳ Kafka

↳ RabbitMQ

8 FIFO

8 LIFO

<u>Cart</u>	<u>Payment</u>	<u>Inventory</u>	<u>Wallet</u>	<u>Email</u>
1. Get → /get All Prod	2. Get → /Create PaymentLink	3. PUT → /reduceItem Count	4. POST → /add Points 5. PUT → /Upgrade Customer	6. POST → /sendEmail



Event Driven Architecture:

o
o
o
o
o
o
o
o

cafe → Khaban Den